

Sobre este plano

Este plano incorpora a crítica de um QA Sênior e foi redesenhado para formar um Quality Engineer moderno — não apenas um testador. A diferença principal: cada projeto herda tudo que veio antes, criando uma curva de aprendizagem em espiral onde você nunca abandona o que aprendeu. Checkpoints a cada 2 semanas garantem consolidação antes de avançar.

80h totais	48 sessões	8 projetos	3 checkpoints
1h40 × 6d × 8sem	seg a sáb	evolutivos	sem. 2, 4 e 6

Visão geral das 8 semanas

Fase	Semanas	Foco	Projeto	Checkpoint
1 — Fundamentos QA	1–2	Teoria, casos de teste, bugs, Git, risco	Auditor G1 → Risk Map PIX	Sem. 2
2 — Automação moderna	3–4	Playwright, POM, fixtures, CI/CD	iFood E2E → Chaos Tester	Sem. 4
3 — APIs e dados	5–6	Postman, SQL, contratos, consistência	API Watchdog → Data Detective	Sem. 6
4 — Quality Engineering + IA	7	Shift Left, observabilidade, performance, IA aplicada	AI QA Copilot	—
5 — Portfólio e mercado	8	GitHub, README, LinkedIn, storytelling técnico	QA Guardian (projeto integrador)	—

Semana 1

Fundamentos que toda vaga cobra

Teoria de testes · Git · Casos de teste · Report de bugs

Novo foco:

Raciocínio de QA

Nesta semana você não escreve uma linha de código de automação. O objetivo é desenvolver o raciocínio de QA: como pensar em testes, como documentar bugs de forma profissional e como organizar seu trabalho com Git desde o primeiro dia. Tudo que aprender aqui aparecerá em todos os projetos seguintes.

Dia	Foco da sessão (1h40)	Exercício prático
Seg	SDLC e STLC: onde o QA entra em cada fase. Tipos de teste: funcional, regressão, exploratório, smoke, sanity. Pirâmide de testes.	Desenhe a pirâmide no papel. Escreva 1 exemplo real de cada nível para um app de banco.
Ter	Como escrever casos de teste profissionais: ID, pré-condição, passos, resultado esperado, resultado obtido, evidência. Caixa preta vs branca.	Escolha o app do seu banco. Escreva 5 casos de teste para o login — inclua 2 casos negativos e 1 caso de borda.
Qua	Como reportar bugs de qualidade: título descritivo, severidade, prioridade, ambiente, passos para reproduzir, evidência (screenshot/log).	Acesse qualquer site público e encontre 2 bugs reais (podem ser de UX). Documente como faria num Jira profissional.
Qui	Git na prática: init, add, commit, push, branch, merge, pull request. Estrutura de repositório para projetos de QA.	Crie o repositório 'qa-portfolio' no GitHub. Faça commit dos casos de teste do dia 2 como arquivo .md com estrutura profissional.
Sex	Risk-based testing: priorizar testes por impacto x probabilidade. Test coverage: o que cobrir primeiro com recursos limitados.	Para o app do banco: monte uma matriz de risco 3x3 (impacto x probabilidade) com 6 funcionalidades. Justifique cada quadrante.
Sab	Projeto da semana — ver abaixo.	Integrar teoria + casos de teste + bugs + Git em projeto real.

PROJETO SEMANA 1 — Alto nível

Auditor do G1 News

Acesse g1.globo.com e aja como auditor de qualidade de um produto real com milhões de usuários diários. Entregáveis: (1) 10 casos de teste escritos ANTES de qualquer automação, cobrindo home, busca e navegação entre categorias — incluindo casos negativos e de borda; (2) matriz de risco das funcionalidades testadas; (3) relatório de bugs encontrados com todos os campos profissionais preenchidos; (4) tudo versionado no GitHub com README explicando a estratégia de teste adotada.

Camada nova: Raciocínio de QA puro — casos de teste, risco e documentação profissional

Por que este projeto: você testa um sistema real de alta escala, não um formulário inventado. Os bugs encontrados são reais. O README vira um relatório de auditoria — exatamente o que o mercado quer ver.

Semana 2

User Stories, Jira e estratégia de testes

Critérios de aceite · Jira · Shift Left · Análise de requisitos

Herda:

Casos de teste · bugs · Git · risco

Novo foco:

Estratégia e Shift Left

A semana 2 aprofunda o lado analítico do QA: como extrair casos de teste de histórias de usuário, como usar o Jira no dia a dia e — o conceito mais moderno — como participar do refinamento de requisitos para prevenir bugs antes de existirem (Shift Left).

Dia	Foco da sessão (1h40)	Exercício prático
Seg	User stories: estrutura, critérios de aceite, definição de pronto (DoD). Como o QA lê uma história e já pensa em testes.	Dada a história: 'Como usuário quero transferir via PIX para pagar contas.' Escreva 7 casos de teste — incluindo race condition e valor zerado.
Ter	Jira na prática: criar projeto Kanban, abrir tickets de bug com todos os campos, workflows, labels, prioridade, sprint.	Crie conta gratuita no Jira. Migre os bugs do projeto G1 (semana 1) para tickets Jira com todos os campos preenchidos corretamente.
Qua	Shift Left: QA participa do refinamento. Como questionar requisitos ambíguos. Perguntas que QA deve fazer antes do dev escrever código.	Pegue 3 histórias de usuário de um produto que você usa. Liste as perguntas que um QA deveria fazer antes do desenvolvimento começar.
Qui	Teste exploratório avançado: session-based testing, charters, tour de Whittaker. Como documentar sem engessar.	Execute uma sessão de 30 minutos de teste exploratório no site books.toscrape.com. Documente usando charter: missão, tempo, achados.
Sex	Revisão e consolidação: organizar o repositório GitHub, melhorar os casos de teste da semana 1 com o que aprendeu.	Refatore os casos de teste do projeto G1 aplicando os critérios de aceite e o Shift Left. Versione com mensagem de commit descritiva.
Sab	Projeto da semana — ver abaixo.	Projeto evolutivo: herda semana 1 + adiciona estratégia e Jira.

PROJETO SEMANA 2 — Alto nível

Risk Map: Auditor de Segurança do PIX

Você recebeu os requisitos de um app fictício de pagamento PIX (crie o documento de requisitos você mesmo). Entregáveis: (1) documento de requisitos com 5 user stories completas com critérios de aceite; (2) lista de perguntas Shift Left que você faria antes do dev começar; (3) matriz de risco completa priorizando o que testar primeiro; (4) 15 casos de teste cobrindo fluxos felizes, negativos, de borda e segurança básica; (5) todos os casos migrados para o Jira com links no README do GitHub.

Herda dos projetos anteriores: Casos de teste · matriz de risco · Git (projeto G1)

Camada nova: Estratégia de testes baseada em risco + Shift Left + Jira

Por que este projeto: nenhum estudante de QA cria requisitos do zero e depois testa em cima deles. Você vai sentir exatamente a dor do time quando os requisitos são ambíguos — e vai aprender a resolver isso antes do código existir.

CHECKPOINT 1

Consolidação — Semanas 1 e 2

O que fazer:

- Revise todos os casos de teste escritos. Estão claros para alguém que nunca viu o sistema?
- Abra o repositório GitHub e leia o README como recrutador. Está profissional?
- Revise os tickets no Jira. Conseguiria um dev reproduzir os bugs sem te perguntar nada?
- Identifique 1 caso de teste fraco em cada projeto e reescreva com melhor cobertura de borda.
- Só avance para a semana 3 quando responder 'sim' para todas as perguntas acima.

Semana 3

Playwright — do zero à arquitetura

Instalação · Locators · Assertions · POM · Fixtures · Debug

Herda:

Casos de teste · risco · Jira · Git

Novo foco:

Automação E2E com Playwright

A automação começa aqui — mas com base sólida. Você não vai 'copiar exemplos': vai entender cada decisão de arquitetura. O Playwright entra como ferramenta para implementar os casos de teste que você já sabe escrever.

Dia	Foco da sessão (1h40)	Exercício prático
Seg	Instalar Playwright + Python + pytest. Browser, Page, Locator, BrowserContext. Diferença entre headed/headless. Primeiro teste rodando.	Escreva um teste que abre o DuckDuckGo, pesquisa 'QA testing' e verifica que a página retornou resultados com o termo buscado.
Ter	Locators modernos: get_by_role, get_by_text, get_by_label, get_by_placeholder, data-testid. Por que evitar XPath e CSS genérico.	Pegue 3 casos de teste do projeto G1 (semana 1). Automatize cada um usando somente locators semânticos — proibido XPath.
Qua	Assertions: expect().to_have_title, to_be_visible, to_have_url, to_have_text, to_be_enabled. Waiters inteligentes vs time.sleep.	Adicione assertions robustas nos testes do dia anterior. Introduza um wait incorreto (sleep fixo), execute 5x e observe a instabilidade.
Qui	Page Object Model: criar classes por página com métodos que representam ações do usuário. Separar lógica de UI dos testes.	Crie uma classe HomePage e uma SearchPage para os testes do G1. Os testes devem ficar sem detalhes de implementação — só regras de negócio.
Sex	Fixtures do pytest: conftest.py, fixture de browser, fixture de página logada. Screenshots automáticos em falhas.	Crie 3 fixtures reutilizáveis no conftest.py. Configure captura automática de screenshot quando qualquer teste falhar.
Sab	Projeto da semana — ver abaixo.	Projeto evolutivo: automação E2E sobre os casos de teste já escritos.

PROJETO SEMANA 3 — Alto nível

iFood QA Suite — Automação de e-commerce real

Automatize o iFood (ifood.com.br) sem login. Implemente em código os casos de teste que você **ESCREVERIA** para esta aplicação. Entregáveis: (1) 8 casos de teste escritos antes da automação (herda metodologia das semanas 1 e 2); (2) suíte Playwright com POM — classe HomePage, SearchPage e RestaurantPage; (3) fixtures no conftest.py para setup/teardown; (4) screenshots automáticos em falhas; (5) tickets Jira para comportamentos inesperados; (6) relatório HTML do pytest com evidências; (7) tudo versionado com PR descritivo no GitHub.

Herda dos projetos anteriores: Casos de teste · POM · matriz de risco · Jira · Git

Camada nova: Automação E2E com Playwright em sistema real de alta complexidade

Por que este projeto: e-commerce real com estado de UI que muda por horário e localização. É exatamente o tipo de sistema caótico que QAs enfrentam no trabalho — nada é previsível.

Semana 4

Playwright avançado e CI/CD

Flaky tests · Paralelismo · GitHub Actions · Debugging profissional

Herda:

POM · fixtures · casos de teste · Jira · Git

Novo foco:

CI/CD e estabilidade de testes

Semana 4 leva os testes para produção: CI/CD rodando automaticamente, testes paralelos e diagnóstico de flaky tests. Você vai deliberadamente quebrar seus testes para aprender a consertá-los.

Dia	Foco da sessão (1h40)	Exercício prático
Seg	GitHub Actions: criar workflow .yml que roda testes a cada push. Entender jobs, steps, artifacts. Ver relatório na aba Actions.	Configure .github/workflows/tests.yml no seu repositório. Faça os testes da semana 3 rodarem no CI. Quebre 1 teste e observe o log de falha.
Ter	Flaky tests: o que são, causas (timing, dados compartilhados, ordem de execução, ambiente). Como diagnosticar com playwright trace.	Introduza 2 flaky tests de propósito (1 com race condition, 1 com dado compartilhado). Execute 10x cada. Diagnostique com trace viewer.
Qua	Corrigir flaky tests: waitFor inteligente, isolamento de dados, retry config. Paralelismo com pytest-xdist.	Corrija os 2 flaky tests do dia anterior. Configure execução paralela e garanta que todos os testes passam em 5 execuções consecutivas.
Qui	Playwright avançado: interceptar requisições de rede (route), mock de API responses, testar estados de erro sem precisar do backend.	Crie um teste que intercepta a chamada de API do iFood e simula um erro 500. Verifique que a UI exibe mensagem de erro adequada.
Sex	Revisão de arquitetura: organização de pastas, nomenclatura de testes, padrões de commit, code review checklist para testes.	Faça code review do seu próprio repositório. Liste 5 melhorias de arquitetura. Implemente pelo menos 3 delas.
Sab	Projeto da semana — ver abaixo.	Projeto evolutivo: suíte robusta com CI/CD e chaos engineering.

PROJETO SEMANA 4 — Alto nível

Chaos Tester — Engenharia do Caos aplicada a QA

Evolua a suíte do iFood (semana 3) para um 'chaos test suite'. Entregáveis: (1) interceptação de 3 chamadas de API com mocks de erro (500, timeout, payload inválido) verificando que a UI se comporta corretamente em cada falha; (2) testes de borda extremos: campo de busca com 500 caracteres, caracteres especiais, SQL injection string, emoji; (3) CI/CD rodando tudo automaticamente no GitHub Actions com relatório de artefatos; (4) execução paralela configurada e documentada; (5) documento 'lessons learned' no README explicando cada flaky test encontrado e como foi resolvido.

Herda dos projetos anteriores: POM iFood · casos de teste · Jira · CI/CD · Git

Camada nova: Chaos engineering + mock de APIs + estabilidade de suíte em produção

Por que este projeto: QA que só testa o caminho feliz não vale nada. Chaos testing é o que separa um QA mediano de um QA que previne incidentes em produção.

CHECKPOINT 2

Consolidação — Semanas 3 e 4

O que fazer:

- O CI/CD está verde? Os testes passam em 5 execuções consecutivas sem intervenção?
- Abra o Playwright Trace Viewer em um teste que falhou. Consegue navegar pelo trace e entender o problema?
- O POM está bem separado? Os testes leem como especificações, não como código?
- Os tickets no Jira estão ligados com os testes automatizados correspondentes?
- Só avance para semana 5 quando o CI estiver verde e os testes forem estáveis.

Semana 5

API Testing com Postman

REST · HTTP · Postman · Contratos · Automação de APIs

Herda:

Casos de teste · Playwright · CI/CD · Git
Jira

Novo foco:

Testes de API profissionais

A maioria dos sistemas modernos é APIs primeiro. Um QA que não testa APIs está cego para metade dos bugs. Esta semana você aprende a testar a camada que ninguém vê — e que é onde os bugs mais críticos se escondem.

Dia	Foco da sessão (3h40)	Exercício prático
Seg	HTTP: verbos (GET, POST, PUT, PATCH, DELETE), status codes (2xx, 3xx, 4xx, 5xx), headers, body, autenticação Bearer/API Key.	Use jsonplaceholder.typicode.com: faça GET /posts, POST /posts, PUT /posts/1 e DELETE /posts/1. Documente status e body de cada um.
Ter	Postman: Collections, variáveis de ambiente, scripts pre-request e de teste (pm.test, pm.expect). Validar schema de resposta.	Crie uma Collection para a API do GitHub (api.github.com). Adicione testes automáticos em cada request validando status, campos obrigatórios e tipos.
Qua	Contrato de API: o que é, por que importa, como validar que o contrato não foi quebrado entre versões. JSON Schema.	Documente o contrato da API do GitHub para o endpoint GET /users/{username}. Escreva um teste Postman que falha se algum campo obrigatório sumir.
Qui	Integrar Postman com Playwright: usar APIRequestContext para testar APIs dentro dos testes E2E. Validar consistência API x UI.	Escreva um teste Playwright que chama a API do GitHub diretamente e verifica que os dados retornados batem com o que é exibido na UI do github.com.
Sex	Autenticação em APIs: Bearer token, OAuth básico, API Key. Como testar cenários de acesso negado (401, 403).	Crie testes Postman que verificam: acesso sem token (401), token inválido (401), token correto (200), endpoint inexistente (404).
Sab	Projeto da semana — ver abaixo.	Projeto evolutivo: API + UI + contratos integrados.

PROJETO SEMANA 5 — Alto nível

API Watchdog — Monitor de qualidade da API do OpenWeather

Use a API gratuita do OpenWeatherMap (openweathermap.org). Entregáveis: (1) Collection Postman com 8 testes automatizados: temperatura de BH, campos obrigatórios presentes, resposta em menos de 2s, CEP inválido retorna 404, unidade metric vs imperial, contrato JSON Schema; (2) testes Playwright que verificam consistência entre dados da API e o que é exibido na UI; (3) casos de teste escritos ANTES da automação (herda metodologia); (4) tickets Jira para qualquer inconsistência API x UI encontrada; (5) CI/CD rodando os testes Playwright automaticamente.

Herda dos projetos anteriores: Casos de teste · Playwright POM · CI/CD · Jira · Git · chaos testing

Camada nova: API testing profissional + consistência API x UI + contrato de API

Por que este projeto: validar consistência entre API e UI é raro no portfólio de candidatos. Demonstra maturidade técnica que vai além do teste de tela.

Semana 6

SQL e validação de dados

SELECT · JOIN · GROUP BY · Integridade relacional · API x banco

Herda:

API testing · Playwright · casos de teste · Jira · Git

Novo foco:

SQL técnico para QA

SQL é o gap mais subestimado por QAs iniciantes. Um bug de dados é invisível na UI mas devastador para o negócio. Saber consultar o banco diretamente separa QA técnico de QA clicador.

Dia	Foco da sessão (1h40)	Exercício prático
Seg	SQL fundamentais para QA: SELECT, WHERE, ORDER BY, LIMIT. Como QA usa SQL para validar dados que a UI exibe.	Use sqliteonline.com com banco demo. Escreva 5 queries: buscar por nome, filtrar por data, ordenar por valor, limitar resultados, buscar nulos.
Ter	JOIN: INNER, LEFT, RIGHT. Quando cada um é usado. Validar integridade relacional entre tabelas.	Escreva queries que encontram: pedidos sem cliente (LEFT JOIN), produtos sem categoria (LEFT JOIN com NULL), usuários sem nenhum pedido.
Qua	GROUP BY, COUNT, SUM, AVG, HAVING. Identificar anomalias de dados: duplicatas, valores fora do range, campos obrigatórios nulos.	Escreva uma query que encontra: clientes com mais de 1 cadastro (duplicata), pedidos com valor zerado, produtos sem estoque mas com pedido ativo.
Qui	Cruzar API x banco: validar que o que a API retorna bate com o que está no banco. Testar integridade de dados em fluxos completos.	Usando a API do Portal da Transparência: chame um endpoint, pegue 5 registros e escreva queries que verificariam se esses dados são consistentes.
Sex	Massa de dados para testes: como criar, como limpar, como isolar. CPF válido/inválido, datas extremas, valores limite.	Crie um script Python que gera 20 registros de massa de teste para um sistema de cadastro de usuários com campos válidos e inválidos.
Sab	Projeto da semana — ver abaixo.	Projeto evolutivo: SQL + API + UI em investigação de dados reais.

PROJETO SEMANA 6 — Alto nível

Data Detective — Investigação forense no Portal da Transparência

Use as APIs públicas do Portal da Transparência (portaldatransparencia.gov.br/api-de-dados). Entregáveis: (1) 10 queries SQL que validariam integridade dos dados retornados pela API (duplicatas, valores ausentes, relacionamentos quebrados); (2) suíte Playwright que chama os endpoints e valida campos obrigatórios, tipos e formato dos dados; (3) relatório de 'anomalias encontradas' documentado como bugs no Jira com severidade e impacto; (4) script de geração de massa de dados para os cenários de teste identificados; (5) seção no README explicando o raciocínio de investigação usado — como um detetive de dados.

Herda dos projetos anteriores: API testing · Playwright · casos de teste · Jira · CI/CD · Git

Camada nova: SQL técnico + investigação de dados reais + integridade relacional

Por que este projeto: dados governamentais reais com APIs documentadas. Nenhum candidato vai ter isso no portfólio. Abre conversa em qualquer entrevista.

O que fazer:

- Você consegue explicar em 2 minutos a diferença entre testar API e testar UI?
- Execute uma query JOIN do zero sem consultar anotações. Conseguiu?
- O projeto Data Detective está linkado ao API Watchdog no GitHub? A evolução é visível?
- Todos os projetos têm CI/CD verde? Todos os bugs estão documentados no Jira?
- Só avance para semana 7 após revisar e melhorar pelo menos 1 projeto anterior.

Semana 7

Quality Engineering + IA aplicada a QA

Observabilidade · Performance · Segurança básica · IA como copiloto

Herda:

Todo o arsenal anterior

Novo foco:

Quality Engineering moderno

IA

Esta semana separa QA clássico de Quality Engineer moderno. Observabilidade, performance básica e segurança entram aqui — mais o módulo mais importante para 2026: usar IA como copiloto real, não como gerador de código cego.

Dia	Foco da sessão (1h40)	Exercício prático
Seg	Observabilidade básica: o que são logs, traces e métricas. Como QA lê logs para diagnosticar bugs que só aparecem em produção.	Acesse qualquer app com DevTools aberto (aba Network + Console). Documente 3 erros ou warnings encontrados nos logs. Interprete cada um.
Ter	Performance testing introdutório: latência, throughput, tempo de resposta. k6 básico: criar script de carga simples.	Instale k6 e escreva um script que faz 10 usuários simultâneos acessarem a API do OpenWeather por 30s. Documente os resultados.
Qua	Segurança básica para QA: autenticação, autorização, IDOR, exposição de dados, SQL injection manual. OWASP Top 10 resumido.	Escolha 1 API pública. Tente: acessar recurso sem autenticação, acessar ID de outro usuário, enviar payload com SQL injection string. Documente.
Qui	IA para QA — parte 1: engenharia de prompt para gerar casos de teste, cenários de borda e massa de dados. Como validar e corrigir o que a IA gera.	Use Claude ou ChatGPT para gerar 10 casos de teste para um fluxo PIX. Avalie cada um: está correto? Está completo? Corrija os que estiverem errados.
Sex	IA para QA — parte 2: usar IA para analisar logs de falha, sumarizar bugs, revisar seletores do Playwright, gerar dados de teste complexos.	Copie um log de erro de um dos seus testes. Peça à IA para analisar e sugerir a causa raiz. Avalie se a sugestão está certa. Documente o processo.
Sab	Projeto da semana — ver abaixo.	Projeto evolutivo: IA como copiloto de toda a suíte anterior.

PROJETO SEMANA 7 — Alto nível

AI QA Copilot — Workflow aumentado por IA

Construa um workflow documentado de como você usa IA no seu processo de QA. Entregáveis: (1) prompt library: 10 prompts testados e refinados para gerar casos de teste, cenários de borda, massa de dados PIX/CPF, análise de logs e revisão de seletores — com exemplos de output bom vs ruim; (2) script k6 de performance rodando contra uma API pública com relatório de resultados; (3) checklist de segurança básica aplicado ao Portal da Transparência (herda semana 6); (4) seção no README 'Como uso IA no meu processo de QA' explicando onde a IA ajuda e onde ela mente; (5) pelo menos 1 caso documentado onde a IA gerou algo errado e como você detectou.

Herda dos projetos anteriores: Todo o arsenal das semanas 1–6

Camada nova: IA como copiloto real + observabilidade + performance básica + segurança

Por que este projeto: a prompt library é um ativo raríssimo no portfólio de QA. Demonstra que você usa IA com pensamento crítico — não cegamente. Isso vale ouro em 2026.

Semana 8

Portfólio, storytelling e mercado

GitHub profissional · LinkedIn · README de nível sênior · Entrevistas

Herda:

Todo o arsenal anterior

Novo foco:

**Projeto integrador +
posicionamento de mercado**

A última semana não é sobre aprender técnica — é sobre comunicar o que você já sabe. O QA Guardian integra tudo em um único repositório que conta uma história profissional completa.

Seg	GitHub profissional: organização de repositórios, pins, contribuições. Como um recrutador lê seu perfil em 60 segundos.	Audite seu GitHub como recrutador. Liste 5 melhorias. Implemente todas: foto, bio, pins dos melhores projetos, README de perfil.
Ter	Como escrever README de nível sênior: problema que resolve, como rodar, decisões técnicas, o que você aprendeu, próximos passos.	Reescreva o README do projeto mais fraco do seu portfólio usando a estrutura: contexto → problema → solução → decisões → aprendizados.
Qua	LinkedIn para QA: headline, about, experiências, skills, projetos. Como descrever projetos pessoais como experiência real.	Atualize seu LinkedIn: adicione os 7 projetos como 'Projetos' com descrição técnica. Cada um com as tecnologias usadas e o que foi testado.
Qui	Preparação para entrevistas baseada em raciocínio: como narrar seus projetos em 2 minutos, como responder 'me fala de um bug interessante'.	Grave um áudio de 2 minutos apresentando o projeto Data Detective. Ouça. Está claro para quem nunca ouviu falar? Refaça até ficar.
Sex	Mock interview: responda por escrito as 15 perguntas da seção de entrevistas deste PDF. Não consulte as respostas — raciocine.	Após responder, compare com o que está no plano. Identifique os 3 pontos mais fracos e crie um plano de revisão para eles.
Sab	PROJETO FINAL — ver abaixo.	QA Guardian: projeto integrador de todo o aprendizado.

PROJETO SEMANA 8 — Alto nível

QA Guardian — Projeto integrador de nível sênior

Crie um repositório público 'qa-guardian' que integra tudo. Entregáveis: (1) suíte Playwright E2E com POM testando o Portal da Transparência (herda semana 6); (2) Collection Postman com 10 testes automatizados das APIs do portal (herda semana 5); (3) queries SQL documentadas para validação de integridade dos dados retornados (herda semana 6); (4) prompt library de IA integrada ao fluxo de trabalho (herda semana 7); (5) CI/CD no GitHub Actions rodando tudo — Playwright + Postman via Newman (herda semana 4); (6) relatório HTML consolidado com evidências de todos os testes; (7) README de nível sênior: contexto, arquitetura de testes, decisões, bugs encontrados reais, lições aprendidas e seção 'O que eu testaria com mais tempo e recursos'; (8) documento de estratégia de testes explicando as decisões de cobertura.

Herda dos projetos anteriores: Tudo das semanas 1–7

Camada nova: Projeto integrador completo — o portfólio que abre portas

Por que este projeto: dados governamentais reais + todas as camadas técnicas + IA integrada. Nenhum candidato QA vai ter algo parecido. É o projeto que você apresenta em qualquer entrevista.

Onde estudar — recursos gratuitos por ferramenta

Playwright (Python)

playwright.dev/python — documentação oficial, melhor fonte
testautomationu.applitools.com — curso gratuito completo
youtube.com — buscar 'Playwright Python pytest 2024'

Git e GitHub

learngitbranching.js.org — aprende Git de forma visual e interativa
docs.github.com/pt — documentação em português
youtube.com — buscar 'Git completo Cod3r PT-BR'

Jira

atlassian.com/software/jira/guides — guia oficial gratuito
atlassian.com/try/cloud/signup — conta gratuita para até 10 usuários
youtube.com — buscar 'Jira tutorial QA PT-BR'

Postman / APIs

learning.postman.com — Postman Academy, completamente gratuito
jsonplaceholder.typicode.com — API fake para praticar
portal.datransparencia.gov.br/api-de-dados — API real documentada

SQL

sqlzoo.net — exercícios interativos, gratuito
sqliteonline.com — banco online para praticar sem instalar
mode.com/sql-tutorial — tutorial visual de SQL

Teoria de Testes

istqb.org — syllabus CTFL em PDF gratuito (em PT-BR)
qacademy.com.br — conteúdo em português sobre QA
ministryoftesting.com — comunidade global de QA

Scrum e Ágil

scrumguides.org — Guia Scrum oficial em PT-BR, gratuito
youtube.com — buscar 'Scrum 9 minutos DevSuperior'
atlassian.com/agile — guia completo

k6 (Performance)

k6.io/docs — documentação oficial com exemplos prontos
youtube.com — buscar 'k6 load testing tutorial'

IA aplicada a QA

Claude (claude.ai) ou ChatGPT — para geração de casos de teste e análise de logs
github.com/features/copilot — GitHub Copilot (gratuito para estudantes)
cursor.sh — IDE com IA integrada, muito usada por QAs

Sites de Prática

saucedemo.com — e-commerce fake, padrão de entrevistas
demoqa.com — site feito para praticar QA
the-internet.herokuapp.com — dezenas de cenários desafiadores
quotes.toscrape.com — listagem e scraping

Vagas e Comunidade

github.com/qa-brasil/vagas — repositório brasileiro de vagas
linkedin.com — buscar 'QA Junior remoto Brasil'
t.me/qabrasil — comunidade Telegram de QA Brasil

Preparação para entrevistas — raciocínio, não script

O objetivo desta seção não é decorar respostas. É desenvolver o raciocínio para responder qualquer pergunta baseando-se nos seus projetos reais. Para cada pergunta abaixo, a melhor resposta sempre começa com: 'No meu projeto X, eu...'

Fundamentos — raciocínio esperado

- Qual a diferença entre teste funcional e não-funcional? Dê um exemplo do seu portfólio de cada.
- Me explique a pirâmide de testes e como você a aplicou nos seus projetos.
- O que é Shift Left? Você praticou isso em algum projeto? Como?
- Qual a diferença entre bug, defeito e falha? Quando cada um acontece no SDLC?
- O que é teste exploratório e quando você usaria em vez de testes roteirizados?
- Como você priorizaria quais funcionalidades testar quando o tempo é curto?

Automação — mostre o código

- Me explique o padrão Page Object Model. Por que você o usou? Quais problemas ele resolve?
- O que é um flaky test? Me conta de um que você encontrou e como resolveu.
- Por que você usa locators semânticos em vez de XPath?
- O que é um fixture no pytest? Me dá um exemplo de quando usou.
- Como você garante que seus testes são independentes entre si?
- O que acontece quando um teste falha no CI? Como você investigaria?

APIs e dados — profundidade técnica

- Qual a diferença entre testar uma API e testar uma UI? Qual é mais importante?
- O que você verifica quando testa um endpoint REST? Me dá um exemplo concreto.
- O que é contrato de API e por que ele importa para QA?
- Como você usaria SQL no dia a dia de QA? Me dá 2 exemplos práticos.
- Me explique um cenário onde a API retornava 200 mas o dado estava errado. Como você detectaria?

Comportamental — mostre maturidade

- Você encontrou um bug crítico na véspera de um release. O que você faz?
- Um desenvolvedor discordou que algo que você reportou é um bug. Como você resolveu?
- Me conta sobre um bug interessante que você encontrou. Por que ele era difícil de reproduzir?
- Como você explicaria para um PM não técnico por que testes automatizados são importantes?
- O que você faria no primeiro mês em uma empresa nova como QA Júnior?

Dica de ouro: leve o link do seu GitHub para a entrevista. Um repositório bem organizado com 8 projetos evolutivos vale mais do que qualquer resposta decorada.